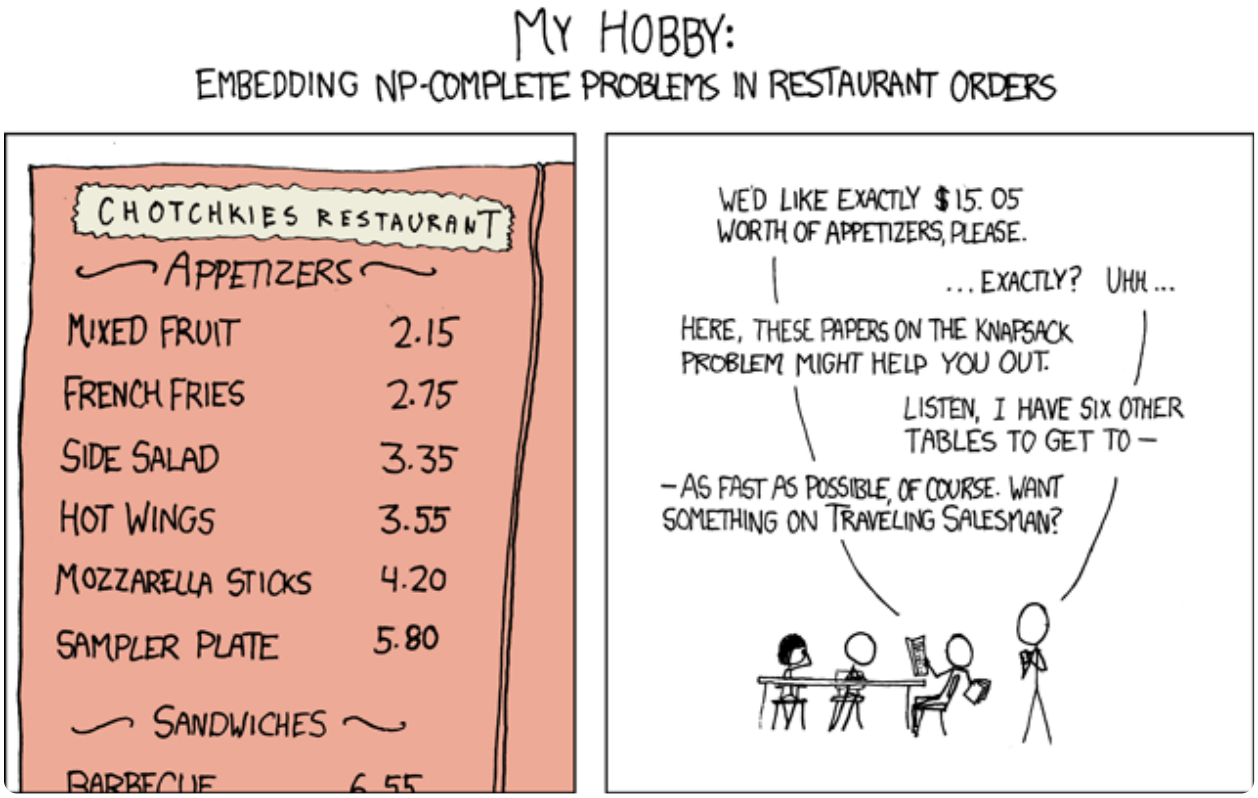
Complexity classes



xkcd #287 (https://xkcd.com/287/)

> If P=NP, then the world would be a profoundly different place than we usually assume it to be. There would be no special value in "creative leaps," no fundamental gap between solving a problem and recognizing the solution once it's found. Everyone who could appreciate a symphony would be Mozart; everyone who could follow a step-by-step argument would be Gauss; everyone who could recognize a good investment strategy would be Warren Buffett.

Scott Aaronson, 2006 (https://scottaaronson.blog/?p=122)

Problems as languages

Recall, how we identified (decision) problems with languages, without loss of generality, we can work over $\Sigma = \{0,1\}$: a decision problem P corresponds to corresponds to a formal language $L_P \subseteq \Sigma^*$ with:
$$w \in L_P \iff \text{$P(w)$ is true}$$

Some problems are algorithmically decidable (e.g. "Does a given DFA accept a given word?"), while others aren't ("Does a given TM halt on itself?").

If a problem is decidable, how much work does it take to find an answer? This is where complexity theory (https://en.wikipedia.org/wiki/Complexity_theory) comes in.

P vs. NP

Complexity theory provides a "zoology" for the computational (in)tractability of problems via complexity classes (https://en.wikipedia.org/wiki/Complexity_class).

Important ones are:

- the class P: all problems solvable by a **deterministic** Turing machine (DTM) in polynomial time,

- the class NP: all problems solvable by a **non-deterministic** Turing machine (NTM) in polynomial time.

What does this amount to?

1. Polynomial time

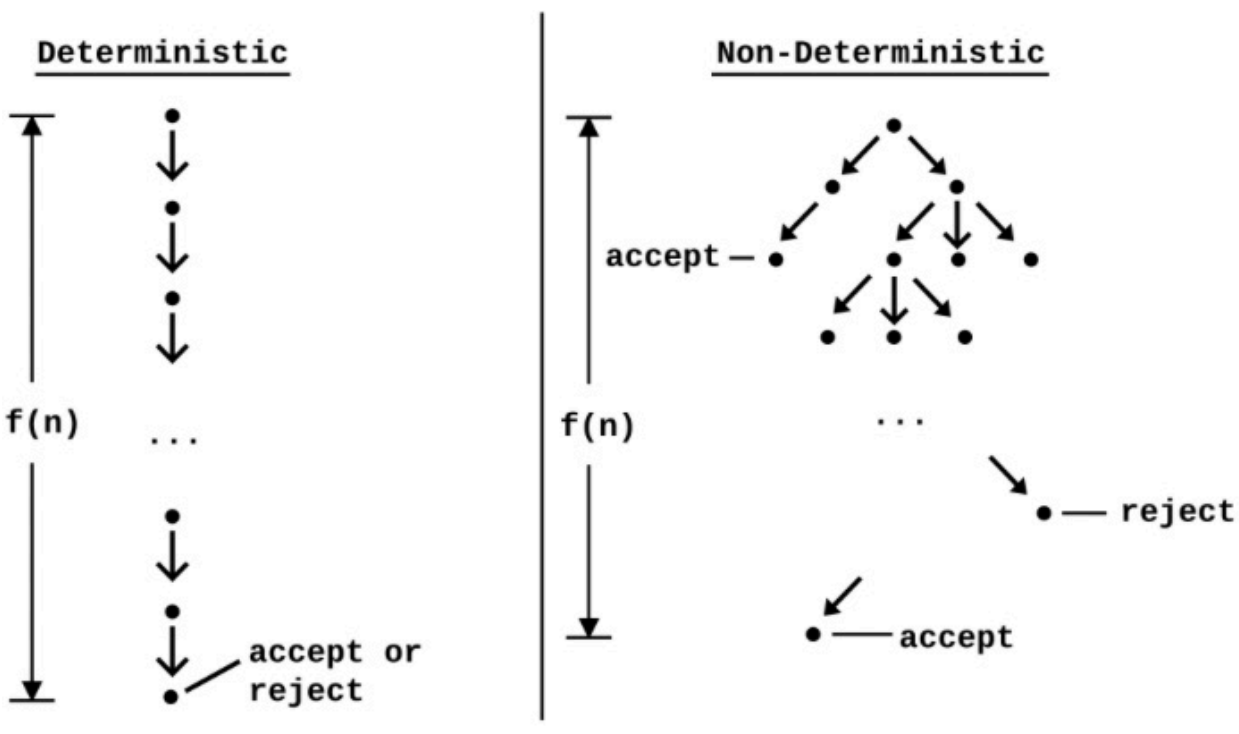
Polynomial time means the class of functions in $\mathcal{O}(n^k)$, for n the input length and some exponent $k \in \mathbb{N}$. The idea is that the runtime should be described by a polynomial function:

$$f(n) = a_k n^k + a_{k-1} n^{k-1} + \ldots + a_1 n + a_0 = \sum_{i=0}^{k} a_i n^i$$

In particular, recall that $\mathcal{O}(\sum_{i=0}^{k} a_i n^i) = \mathcal{O}(a_k n^k) = \mathcal{O}(n^k)$.

2. DTMs vs. NTMs

Intuitively, while a DTM has no freedom of choice for its next moves, an NTM does. Moreover, an NTM can "guess" the next move for an exponential number of possible transitions, check each of them in polynomial time "in parallel":



Formally, we define:

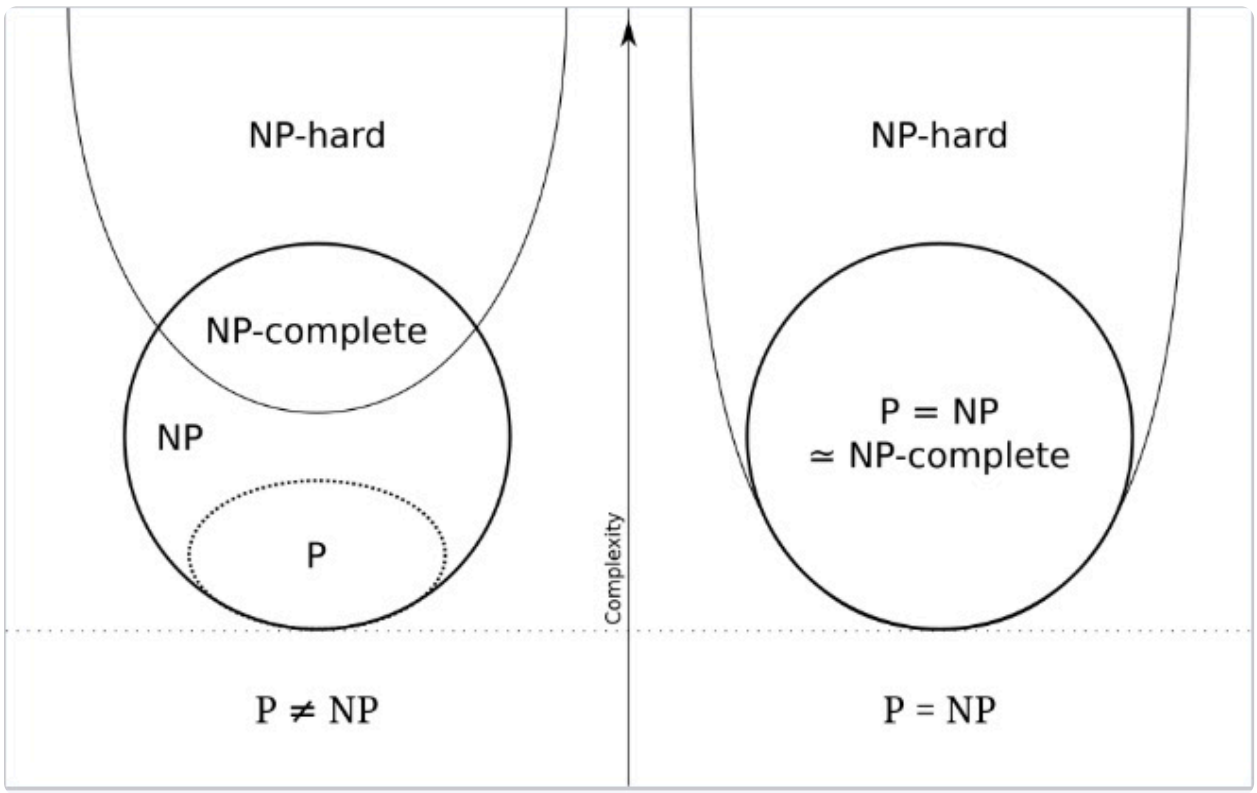
$$\mathrm{P} = \{L \subseteq \{0,1\}^* \mid \text{there is a DTM } \mathcal{M} \text{ with } L(\mathcal{M}) = L \text{ and runtime } T(\mathcal{M}) \in \mathcal{O}$$

$$\mathrm{P} = \{L \subseteq \{0,1\}^* \mid \text{there is an NTM } \mathcal{M} \text{ with } L(\mathcal{M}) = L \text{ and runtime } T(\mathcal{M}) \in$$

Any DTM is a special kind of NTM (one where no actual choice is possible), thus $\mathrm{P} \subseteq \mathrm{NP}$. What about the reverse inclusion: does $\mathrm{NP} \overset{?}{\subseteq} \mathrm{P}$, hence $\mathrm{NP} \overset{?}{=} \mathrm{P}$?

This is a long-standing open problem, see S. Aaronson's quote above!

Empirically, this is generally expected to be **false**, but to this day we don't have a proof!



Intuitively, problems in P are quick to solve, while problems in NP are quick to **check**. They tend to be not quick to solve, which is why they are thought of as **intractable** problems, meaning inefficient to solve.

Examples of problems

problems in P	**problems in NP**
sorting	traveling salesperson (TSP)
Rubik's cube	Integer factorization (see cryptography)
multiplication	Sudoku
...	Protein folding
...	...

Reduction of problems and NP-completeness

How to show that some L is in NP? The idea is to use a *reduction argument*: Given problems/languages L_1, L_2 with $L_1 \notin \mathrm{P}$ we could try to **reduce** L_1 to L_2, then $L_2 \notin \mathrm{P}$. We want the **reduction** to be be doable by a polynomial-time algorithm.

If there exists such a reduction from L_1 to L_2, we write $L_1 \leq_{\mathrm{P}} L_2$.

Among the problems in NP, the NP**-complete** problems have a special role:

Definition (NP-completeness). We call a problem L NP**-complete** if:

1. $L \in \mathrm{NP}$
2. For every $L' \in \mathrm{NP}$ we have $L' \leq_{\mathrm{P}} L$ (we say, L' is NP**-hard**).

Theorem. Assume that P_1 is NP-complete, P_2 is in NP, and $P_1 \leq_{\mathrm{P}} P_2$, then P_2 is also NP-complete.

SAT

A fundamental NP-complete problem is SAT, the **boolean satisfiability problem**. It asks: given a formula from boolean propositional logic, is it satisfiable?

Boolean propositional logic

Boolean propositional formulas can be used to model logic states of systems such as circuits, networks, databases,...

We work over an (infinite) alphabet Var of propositional variables $p, q, r, \ldots$ that can take the values "true" and "false", denoted by 1 and 0, resp.

Definition (Boolean formulas).
A **formula** is an expression of one of the following forms:

1. $\varphi \vee \psi$ (**disjunction/"or"**), for formulas φ, ψ
2. $\varphi \wedge \psi$ (**conjunction/"and"**), for formulas φ, ψ
3. $\neg\varphi$ (**negation/"not"**)
4. p (**variables**)
5. $\mathrm{b} \in \{0, 1\}$ (**truth values**)

Definition (Valuation).
A **valuation** or **assignment** is a function $\mathfrak{J} : \mathrm{Var} \to \{0, 1\}$. Any valuation acts on the formulas by defining:

1. $\mathfrak{J}(\varphi \vee \psi) = \max(\mathfrak{J}(\varphi), \mathfrak{J}(\psi))$
2. $\mathfrak{J}(\varphi \wedge \psi) = \min(\mathfrak{J}(\varphi), \mathfrak{J}(\psi))$
3. $\mathfrak{J}(\neg\varphi) = 1 - \mathfrak{J}(\varphi)$
4. $\mathfrak{J}(p) = p$
5. $\mathfrak{J}(\mathrm{b}) = \mathrm{b}$

The truth tables for the logical connectives are given as follows:

p	q	$p \vee q$	$p \wedge q$	$\neg p$
0	0	0	0	1
0	1	1	0	1
1	0	1	0	0
1	1	1	1	0

We define **implication** by $\varphi \to \psi := (\neg\varphi \vee \psi)$, and **equivalence** by $\varphi \leftrightarrow \psi := (\varphi \to \psi) \wedge (\psi \to \varphi)$.

Example.
Define the evaluation $\mathfrak{J}$ by $\mathfrak{J}(p) := 0$ and $\mathfrak{J}(q) := 1$. Consider the formulas:

- $\varphi := \neg p \vee q$
- $\psi := \neg p \wedge (p \vee \neg q)$

Under $\mathfrak{J}$ the formulas evaluate to:

- $\varphi^{\mathfrak{J}} = \max(1, 1) = 1$
- $\psi^{\mathfrak{J}} = \min(1, 0) = 0$

The formula $\xi := p \wedge (\neg p \vee q) \wedge \neg q$ is, in fact, **not** satisfiable, in the sense that there exists no assignment $\mathfrak{J}$ such that $\xi^{\mathfrak{J}} = 1$ (why?).

The problem SAT is defined as the language of (the codings of) all formulas φ which are **satisfiable**, i.e., for which there exists an assignment $\mathfrak{J}$ such that $\varphi^{\mathfrak{J}} = 1$

Theorem (Cook–Levin).
SAT is NP-complete.

SAT-solvers are used to solve a range of various problems (https://en.wikipedia.org/wiki/SAT_solver#Applications). Because SAT is known to only be in NP rather than P, it is a generally infeasible problem. Hence, developing efficient SAT-solvers (https://en.wikipedia.org/wiki/SAT_solver) is an active and challenging area.

If one could show that SAT were in P, then this would show (https://en.wikipedia.org/wiki/P_versus_NP_problem) $\mathrm{P} = \mathrm{NP}$; a Millenium problem (https://en.wikipedia.org/wiki/Millennium_Prize_Problems) with a $1M reward, and many

consequences for questions of algorithm efficiency across the board!

Conjunctive normal form

For algorithmic purposes such as SAT-solving, it is often convenient to assume the input formula to be in a special format, called **conjunctive normal form (CNF)** (https://en.wikipedia.org/wiki/Conjunctive_normal_form), i.e., a conjunction of disjunctions of literals ℓ_{ij}:

$$\varphi = \bigwedge_i \bigvee_j \ell_{i,j},$$

where a literal ℓ_{ij} is either a variable p or the negation $\neg p$ of a variable p.

For instance, the formula $\psi = \neg p \wedge (p \vee \neg q)$ is in CNF. Another example is $\vartheta := (p \vee \neg q) \wedge (\neg p \vee q \vee r) \wedge p$.

One can show that every boolean formula can be brought into CNF preserving the set of satisfying assignments.